REGRESSION

Importing the required libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
# Reading the data from input csv file into a Pandas DataFrame
computers = pd.read_csv("datasets.zip")
# Printing the first 5 rows
computers.head()
```

	Units	Minutes
0	1	23
1	2	29
2	3	49
3	4	64
4	4	74

Calculating the mean

```
mean_value = computers['Minutes'].mean()
print(mean_value)
```

```
97.21428571428571
```

Plotting the actual time taken for various values of number of faulty units.

```
plt.scatter(computers['Units'],computers['Minutes'],color='blue', marker='o')
```

```
<matplotlib.collections.PathCollection at 0x7f91c8f295d0>
```

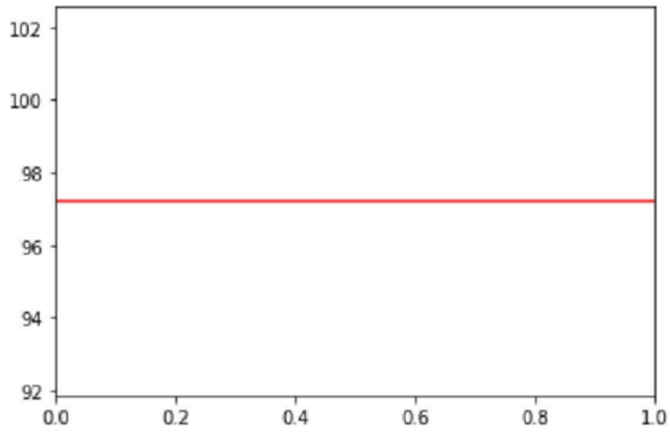


Plotting the mean



```
1 plt.axhline(y=mean_value,c="r")
```

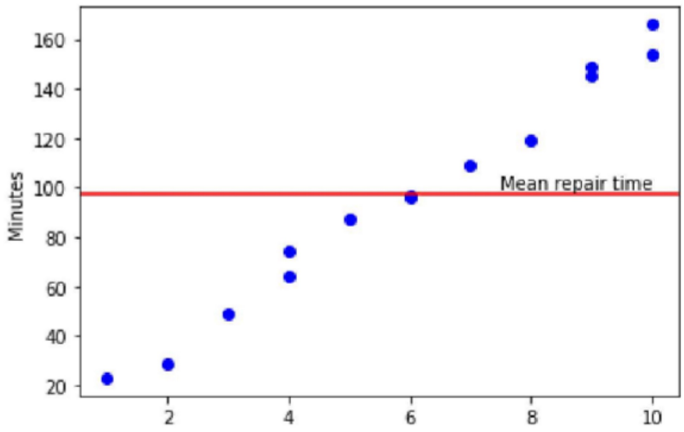
```
<matplotlib.lines.Line2D at 0x7f91c8a11910>
```



Adding annotation, xlabel and ylabel.

```
1 plt.annotate("Mean repair time",xy = (7.5,mean_value+2))
2 plt.xlabel("Units")
3 plt.ylabel("Minutes")
```

```
Text(0, 0.5, 'Minutes')
```



Best Fit Model

```
1 X = computers[['Units']]
2 y=computers['Minutes']
3 from sklearn.linear_model import LinearRegression
4 model = LinearRegression()
5 model.fit(X, y)
6 print("Intercept:", model.intercept_)
7 print("Coefficients:",model.coef_)
```

```
Intercept: 4.161654135338296
Coefficients: [15.50877193]
```

```
1 x = computers.Units
2 y = computers.Minutes
3 xiyi = x*y
4 n = len(computers)
5 xmean = computers.Units.mean()
6 ymean = computers.Minutes.mean()
7 numerator = xiyi.sum() - n*xmean*ymean
8 denominator = (x**2).sum() - n*(xmean**2)
9 m = numerator/denominator
10 c = ymean - (m*xmean)
11 print('intercept : ',c,' coefficient : ',m)
```

```
intercept :  4.161654135338296  coefficient :  15.508771929824569
```

```
1 min_best_fit_model = c + m*computers.Units
2 computers['min_best_fit_model'] = min_best_fit_model
3 print(min_best_fit_model)
```

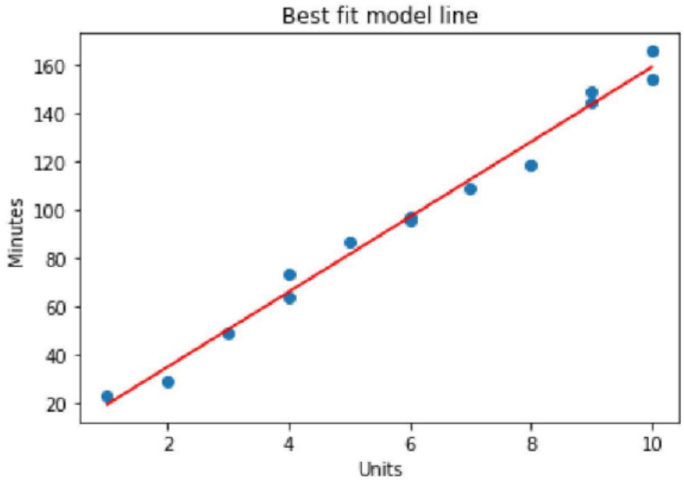
```
0      19.670426
1      35.179198
2      50.687970
3      66.196742
```

```
4      66.196742
5      81.705514
6      97.214286
7      97.214286
8     112.723058
9     128.231830
10    143.740602
11    143.740602
12    159.249373
13    159.249373
Name: Units, dtype: float64
```

Validation of best fit model

```
computers[["Units","Minutes","min_best_fit_model"]]
fig,ax = plt.subplots()
ax.scatter(x="Units",y="Minutes",data=computers)
ax.plot(computers.Units,computers.min_best_fit_model,color="red")
ax.set_ylabel("Minutes")
ax.set_xlabel("Units")
ax.set_title("Best fit model line")
```

```
Text(0.5, 1.0, 'Best fit model line')
```



Computing the SSE for the best fit model

```
best_fit_model_obs = pd.DataFrame({"Units":computers.Units,
              "Actual time":computers.Minutes,
              "Predicted time":computers.min_best_fit_model,
              "Error":computers.min_best_fit_model - computers.Minutes})

best_fit_model_obs
```

	Units	Actual time	Predicted time	Error
0	1	23	19.670426	-3.329574
1	2	29	35.179198	6.179198
2	3	49	50.687970	1.687970
3	4	64	66.196742	2.196742
4	4	74	66.196742	-7.803258
5	5	87	81.705514	-5.294486
6	6	96	97.214286	1.214286
7	6	97	97.214286	0.214286
8	7	109	112.723058	3.723058
9	8	119	128.231830	9.231830
10	9	149	143.740602	-5.259398
11	9	145	143.740602	-1.259398
12	10	154	159.249373	5.249373
13	10	166	159.249373	6.750627

```
1 sum(best_fit_model_obs.Error**2)
```

```
348.848370927318
```

Solving Least Squares Regression using Closed Form Linear Equation

```
1 X = np.matrix([np.ones(computers.shape[0]),computers["Units"]]).T
2 y = np.matrix(computers["Minutes"]).reshape(-1,1)
3 beta = (np.linalg.pinv(X.T*X)*X.T)*y
4 beta
```

```
matrix([[ 4.16165414],
        [15.50877193]])
```

Coefficient of Determination

```
1 SST = sum((computers.Minutes.mean() - computers.Minutes)**2)
2 SST
```

```
27768.357142857145
```

```
1 SSE = sum(best_fit_model_obs.Error**2)
2 SSE
```

```
348.848370927318
```

```
SSR = SST - SSE
SSR
```

```
27419.508771929828
```

```
Rsq = SSR/SST
Rsq
```

```
0.9874371980620736
```

```
Rsq1 = model.score(computers[["Units"]],y)
Rsq1
```

```
/usr/local/lib/python3.7/dist-packages/sklearn/utils/validation.py:598: FutureWarning: r
  FutureWarning,
0.9874371980620736
```

DECISION TREE ON CREDIT RISK DATA SET

Import data set

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
credit_data = pd.read_csv("creditriskdataset.zip")
credit_data.head()
```

	over_draft	credit_usage	credit_history	purpose	current_balance	Average_
0	<0	6	critical/other existing credit	radio/tv	1169	

```
1 credit_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 21 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   over_draft              1000 non-null   object
 1   credit_usage            1000 non-null   int64
 2   credit_history          1000 non-null   object
 3   purpose                 1000 non-null   object
 4   current_balance         1000 non-null   int64
 5   Average_Credit_Balance  1000 non-null   object
 6   employment              1000 non-null   object
 7   location                1000 non-null   int64
 8   personal_status         1000 non-null   object
 9   other_parties           1000 non-null   object
 10  residence_since         1000 non-null   int64
 11  property_magnitude      1000 non-null   object
 12  cc_age                  1000 non-null   int64
 13  other_payment_plans     1000 non-null   object
 14  housing                 1000 non-null   object
 15  existing_credits        1000 non-null   int64
 16  job                     1000 non-null   object
 17  num_dependents          1000 non-null   int64
 18  own_telephone           1000 non-null   object
 19  foreign_worker          1000 non-null   object
 20  class                   1000 non-null   object
dtypes: int64(7), object(14)
memory usage: 164.2+ KB
```

Setting the predictors and target and Encoding the categorical values

```
1 # Selecting the predictor attributes
2 X = credit_data.columns.drop("class")
3 # Selecting  the target
4 y = credit_data['class']
5 credit_data_encoded = pd.get_dummies(credit_data[X])
6 print("Total number of predictors after encoding = ", len(credit_data_encoded.columns))
7 credit_data_encoded.columns
```

```
Total number of predictors after encoding =  61
Index(['credit_usage', 'current_balance', 'location', 'residence_since',
       'cc_age', 'existing_credits', 'num_dependents', 'over_draft_0<=X<200',
       'over_draft_<0', 'over_draft_>=200', 'over_draft_no checking',
       'credit_history_all paid',
       'credit_history_critical/other existing credit',
       'credit_history_delayed previously', 'credit_history_existing paid',
```

```
       'credit_history_no credits/all paid', 'purpose_business',
       'purpose_domestic appliance', 'purpose_education',
       'purpose_furniture/equipment', 'purpose_new car', 'purpose_other',
       'purpose_radio/tv', 'purpose_repairs', 'purpose_retraining',
       'purpose_used car', 'Average_Credit_Balance_100<=X<500',
       'Average_Credit_Balance_500<=X<1000', 'Average_Credit_Balance_<100',
       'Average_Credit_Balance_>=1000',
       'Average_Credit_Balance_no known savings', 'employment_1<=X<4',
       'employment_4<=X<7', 'employment_<1', 'employment_>=7',
       'employment_unemployed', 'personal_status_female div/dep/mar',
       'personal_status_male div/sep', 'personal_status_male mar/wid',
       'personal_status_male single', 'other_parties_co applicant',
       'other_parties_guarantor', 'other_parties_none',
       'property_magnitude_car', 'property_magnitude_life insurance',
       'property_magnitude_no known property',
       'property_magnitude_real estate', 'other_payment_plans_bank',
       'other_payment_plans_none', 'other_payment_plans_stores',
       'housing_for free', 'housing_own', 'housing_rent',
       'job_high qualif/self emp/mgmt', 'job_skilled',
       'job_unemp/unskilled non res', 'job_unskilled resident',
       'own_telephone_none', 'own_telephone_yes', 'foreign_worker_no',
       'foreign_worker_yes'],
      dtype='object')
```

Splitting the data into train and test sets

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(credit_data_encoded, y,test_size=0.15,r
print("Shape of X_train and y_train are:", X_train.shape, "and", y_train.shape, " respec
print("Shape of X_test and y_test are:", X_test.shape, "and", y_test.shape, " respective
```

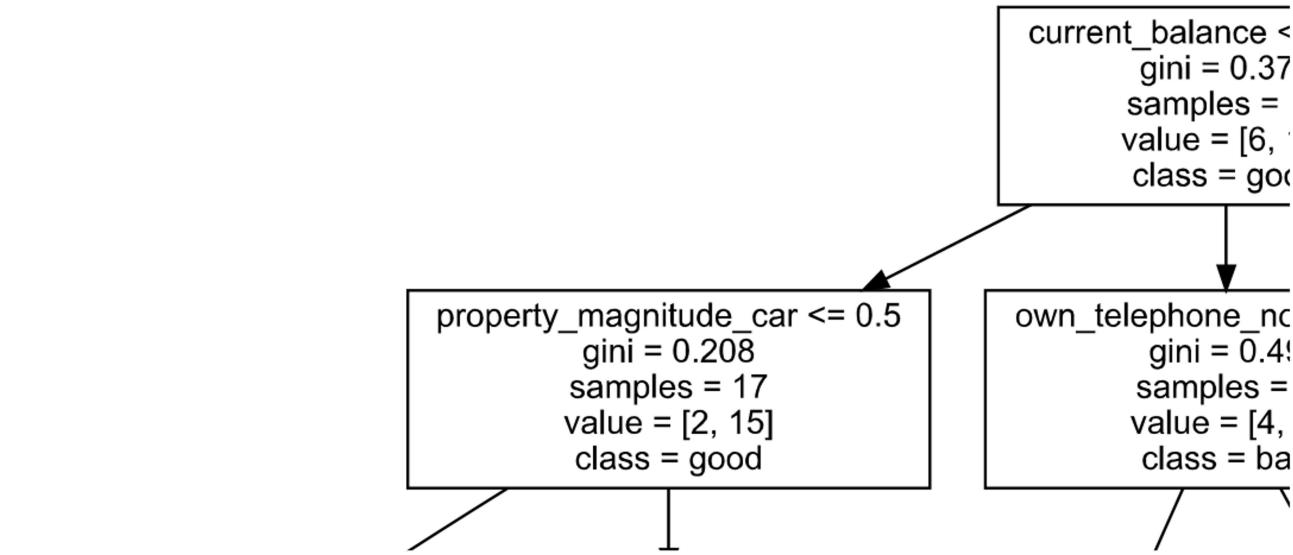
```
Shape of X_train and y_train are: (850, 61) and (850,)  respectively
Shape of X_test and y_test are: (150, 61) and (150,)  respectively
```

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(random_state = 1)
model.fit(X_train,y_train)
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)
```

Visualizing the decision tree

```
from sklearn.tree import export_graphviz
import graphviz
# Generating the tree
dot_data = export_graphviz(model, out_file=None,
                           feature_names=credit_data_encoded.columns,
                           class_names=model.classes_,
                           )
graph = graphviz.Source(dot_data)
```

```
9  graph
```



Evaluating Performance of a model

```
1 train_accuracy = model.score(X_train,y_train)
2 print("Accuracy of the model on train data = ",train_accuracy)
3 test_accuracy = model.score(X_test,y_test)
4 print("Accuracy of the model on test data = ",test_accuracy)
```

```
Accuracy of the model on train data =  1.0
Accuracy of the model on test data =  0.6533333333333333
```

```
1 model1 = DecisionTreeClassifier(min_samples_split=10,min_impurity_decrease=0.005)
2 model1.fit(X_train,y_train)
3 print("train_accuracy = ", model1.score(X_train,y_train))
4 print("test_accuracy = ", model1.score(X_test,y_test))
```

```
train_accuracy =  0.7635294117647059
test_accuracy =  0.7266666666666667
```

```
model2 = DecisionTreeClassifier(min_samples_split=20,min_impurity_decrease=0.1)
# Fitting the model to the training data
model2.fit(X_train,y_train)
# Measuring the accuracy of the model
print("Model2 train accuracy = ", model2.score(X_train,y_train))
print("Model2 test accuracy = ", model2.score(X_test,y_test))
```

```
Model2 train accuracy =  0.6976470588235294
Model2 test accuracy =  0.7133333333333334
```

✓ 0s completed at 1:17 PM